

DSC 190/291 Assignment 5

Detailed Write-Up

Dhruv Patel

May 12, 2026

Contents

1	Part A: Sparse Linear Predictors — Statistical Sparsity versus Agnostic Hardness	2
2	Part B: Convex Fixed-Feature Learning and Its Limits	12
3	Part C: AI Usage Report	27

1. Part A: Sparse Linear Predictors — Statistical Sparsity versus Agnostic Hardness

This problem continues the sparse-linear-model theme from Homework 4. There you studied sparse halfspaces through VC dimension, structural risk minimization, validation, and brute-force search over supports. Here you will see what changes in the agnostic proper setting: the statistical story survives, but the empirical 0-1 optimization problem can become computationally hard when the sparsity level is part of the input.

For $d \in \mathbb{N}$ and $k \leq d$, define the class of k -sparse homogeneous linear classifiers

$$\mathcal{H}_{d,k} = \{ h_w : x \mapsto \text{sign}(\langle w, x \rangle) : w \in \mathbb{R}^d, \|w\|_0 \leq k \},$$

where throughout this problem

$$\text{sign}(t) = +1$$

if $t \geq 0$, and $\text{sign}(t) = -1$ otherwise.

For a labeled sample $S = ((x_1, y_1), \dots, (x_m, y_m))$, define

$$\text{AGREEMENT}_{\mathcal{H}_{d,k}}(S, K) = 1$$

iff there exists $h_w \in \mathcal{H}_{d,k}$ that correctly labels at least K examples in S .

1. (7 points) What Homework 4 already gives you

Recall from Homework 4 that

$$\text{VCdim}(\mathcal{H}_{d,k}) \leq O\left(k \log \frac{ed}{k}\right)$$

for $1 \leq k \leq \frac{d}{2}$. For larger k , the usual dense-halfspace bound gives $\text{VCdim}(\mathcal{H}_{d,k}) \leq O(d)$. Do not reprove this bound. Explain what it implies for agnostic sample complexity if exact empirical risk minimization over $\mathcal{H}_{d,k}$ were computationally available.

Also recall the brute-force realizable algorithm from Homework 4: enumerate supports $T \subseteq [d]$ with $|T| \leq k$, and for each support solve a linear feasibility problem. State its runtime up to polynomial factors in the feasibility solver. In which regimes of k is this polynomial in d ? Explain why this is a statistical/computational distinction rather than a contradiction.

Agnostic sample complexity under a computational oracle.

The important point from Homework 4 is that sparse halfspaces have much smaller VC dimension than arbitrary dense halfspaces when $k \ll d$. The VC-dimension bound says that the statistical complexity of the class is controlled by

$$V = O\left(k \log \frac{ed}{k}\right)$$

for $1 \leq k \leq d/2$. In the agnostic PAC setting, a standard uniform convergence theorem says that a class with VC dimension V can be learned to excess error ε and confidence $1 - \delta$ using

$$m(\varepsilon, \delta) = O\left(\frac{V + \log(1/\delta)}{\varepsilon^2}\right)$$

samples, assuming we can perform exact empirical risk minimization over the class.

Therefore, if exact ERM over $\mathcal{H}_{d,k}$ were computationally available, then for $1 \leq k \leq d/2$ the agnostic sample complexity would be

$$m(\varepsilon, \delta) = O\left(\frac{k \log(ed/k) + \log(1/\delta)}{\varepsilon^2}\right).$$

For larger k , the dense-halfspace bound gives

$$m(\varepsilon, \delta) = O\left(\frac{d + \log(1/\delta)}{\varepsilon^2}\right).$$

This means that, statistically, sparsity helps a lot. If k is small compared to d , then the number of samples needed depends mainly on $k \log(ed/k)$ rather than on d . For example, if $k = O(1)$, then the sample complexity is only logarithmic in d up to constants and accuracy terms.

Brute-force realizable algorithm and runtime.

The brute-force algorithm from Homework 4 works as follows. First, enumerate every possible support

$$T \subseteq [d]$$

with

$$|T| \leq k.$$

For each support T , restrict the classifier to only use coordinates in T . Then solve a linear feasibility problem to check whether there exists a vector supported on T that correctly classifies the sample.

The number of possible supports is

$$\sum_{i=0}^k \binom{d}{i}.$$

Using the standard bound

$$\sum_{i=0}^k \binom{d}{i} \leq \left(\frac{ed}{k}\right)^k,$$

the runtime is

$$\text{Runtime} = \left(\frac{ed}{k}\right)^k \cdot \text{poly}(m, d)$$

up to polynomial factors from the feasibility solver. More precisely, for each support, the feasibility problem only has k active variables, but the full input still has m examples and d possible coordinates, so it is reasonable to write the non-combinatorial part as a polynomial factor.

When is this polynomial in d ?

If $k = O(1)$, then

$$\left(\frac{ed}{k}\right)^k = d^{O(1)},$$

so the brute-force algorithm is polynomial in d .

If $k = O(\log d)$, then the runtime becomes roughly

$$d^{O(\log d)},$$

which is quasi-polynomial, not polynomial.

If k grows like a power of d , for example $k = d^\alpha$ for some $\alpha > 0$, then the runtime becomes exponential or super-polynomial in d .

Why this is not a contradiction.

This is a statistical/computational distinction. The VC-dimension result says that the class does not require too many samples to learn, assuming an ERM oracle is available. However, the brute-force algorithm shows that actually finding the empirical risk minimizer may require searching through exponentially many supports.

So the statistical problem and the computational problem are different. Sparsity makes the class statistically simpler because there are fewer effective degrees of freedom, but it can make the optimization problem computationally difficult because the learner must identify which k coordinates matter. Part A.2 shows that this difficulty is not just a weakness of brute force; it is connected to NP-hardness.

2. (24 points) Hardness of agreement via set cover.

Reduce SETCOVER to AGREEMENT $_{\mathcal{H}_{d,k}}$. A SETCOVER instance consists of a universe $U = \{u_1, \dots, u_r\}$, sets $A_1, \dots, A_q \subseteq U$, and an integer $k \leq q$. The question is whether there exists $R \subseteq [q]$ with $|R| \leq k$ and $\bigcup_{j \in R} A_j = U$.

Construct a labeled sample and threshold K such that the set-cover instance is a yes-instance iff there is a k -sparse homogeneous linear classifier that correctly labels at least K examples. The sample may be treated as a multiset, so repeated examples count with multiplicity.

Prove both directions of the reduction.

Then invoke the Week 5 learner-to-agreement theorem. Explain in two or three sentences how it applies to the family $\mathcal{H}_{d,k}$ when both d and k are part of the input. Conclude that, if $\text{NP} \neq \text{RP}$, there is no efficient proper agnostic PAC learner for sparse linear classifiers when k is part of the input.

Reduction construction.

We reduce an arbitrary SETCOVER instance to an instance of AGREEMENT $_{\mathcal{H}_{d,k}}$. Let the set-cover instance be

$$U = \{u_1, \dots, u_r\}, \quad A_1, \dots, A_q \subseteq U, \quad k \leq q.$$

We create one coordinate for each set A_j . Therefore set

$$d = q.$$

The goal is to make negative coordinates of w correspond to selected sets.

Let

$$M = q + 1.$$

This repetition parameter makes covering every universe element more important than gaining or losing all positive examples combined.

Positive examples.

For every set index $j \in [q]$, define

$$x^{(j)} = -e_j \in \mathbb{R}^q, \quad y^{(j)} = +1.$$

Here e_j is the j -th standard basis vector. Thus $x^{(j)}$ has value -1 in coordinate j and 0 in every other coordinate.

These positive examples are used to make negative weights safe. If $w_j < 0$, then

$$\langle w, x^{(j)} \rangle = \langle w, -e_j \rangle = -w_j > 0,$$

so the positive example is correctly classified as $+1$.

Negative examples.

For every universe element $u_i \in U$, define a vector $z^{(i)} \in \mathbb{R}^q$ by

$$z_j^{(i)} = \begin{cases} 1, & u_i \in A_j, \\ 0, & u_i \notin A_j. \end{cases}$$

This vector records which sets contain the element u_i .

Give this example label

$$y = -1.$$

Include this same negative example with multiplicity

$$M = q + 1.$$

So each universe element contributes M repeated negative examples.

Threshold.

There are q positive examples and Mr negative examples. Set

$$K = q + Mr = q + (q + 1)r.$$

This is the total number of examples in the constructed sample. Therefore, achieving agreement at least K means correctly classifying every example.

How the construction encodes set cover.

A negative coordinate $w_j < 0$ means that set A_j is selected. Define

$$R = \{j \in [q] : w_j < 0\}.$$

Since w is k -sparse,

$$|R| \leq \|w\|_0 \leq k.$$

Thus any k -sparse classifier can select at most k sets.

For a universe element u_i , the score of its negative example is

$$\langle w, z^{(i)} \rangle = \sum_{j: u_i \in A_j} w_j.$$

If some selected set A_j contains u_i , then at least one term in the sum is negative. In the construction we will use weights $w_j = -1$ for selected sets and $w_j = 0$ otherwise, so the score becomes negative exactly when u_i is covered.

Since the negative label is -1 , a negative example is correctly classified precisely when

$$\langle w, z^{(i)} \rangle < 0.$$

Thus the negative examples reward covered universe elements.

Forward direction: set cover yes-instance $\Rightarrow$ agreement yes-instance.

Assume the set-cover instance is a yes-instance. Then there exists a subset

$$R \subseteq [q]$$

such that

$$|R| \leq k$$

and

$$\bigcup_{j \in R} A_j = U.$$

Define a weight vector $w \in \mathbb{R}^q$ by

$$w_j = \begin{cases} -1, & j \in R, \\ 0, & j \notin R. \end{cases}$$

Then

$$\|w\|_0 = |R| \leq k,$$

so $h_w \in \mathcal{H}_{d,k}$.

Now check the positive examples. For each $j \in [q]$,

$$\langle w, x^{(j)} \rangle = \langle w, -e_j \rangle = -w_j.$$

If $j \in R$, then $w_j = -1$, so

$$-w_j = 1 > 0.$$

Hence the prediction is $+1$, which is correct.

If $j \notin R$, then $w_j = 0$, so

$$-w_j = 0.$$

By the convention in Part A,

$$\text{sign}(0) = +1.$$

Hence this positive example is also correctly classified.

Therefore all q positive examples are classified correctly.

Now check the negative examples. Since R is a set cover, every $u_i \in U$ belongs to at least one selected set A_j with $j \in R$. Therefore, for each i ,

$$\langle w, z^{(i)} \rangle = \sum_{j: u_i \in A_j} w_j \leq -1 < 0.$$

Hence

$$\text{sign}(\langle w, z^{(i)} \rangle) = -1,$$

which is the correct label. Since each negative example is repeated M times, all Mr repeated negative examples are correctly classified.

Therefore the total number of correctly classified examples is

$$q + Mr = K.$$

Thus

$$\text{AGREEMENT}_{\mathcal{H}_{d,k}}(S, K) = 1.$$

Reverse direction: agreement yes-instance $\Rightarrow$ set cover yes-instance.

Now assume

$$\text{AGREEMENT}_{\mathcal{H}_{d,k}}(S, K) = 1.$$

Then there exists a vector $w \in \mathbb{R}^q$ with

$$\|w\|_0 \leq k$$

such that h_w correctly labels at least

$$K = q + Mr$$

examples.

But the constructed sample has exactly $q + Mr$ examples total. Therefore, correctly labeling at least K examples means correctly labeling every single example.

Define

$$R = \{j \in [q] : w_j < 0\}.$$

Since w has at most k nonzero coordinates,

$$|R| \leq \|w\|_0 \leq k.$$

So R selects at most k sets.

We now prove that these selected sets cover all of U .

Take any universe element $u_i \in U$. Its negative example $z^{(i)}$ is correctly classified. Since the label is -1 , correctness requires

$$\text{sign}(\langle w, z^{(i)} \rangle) = -1.$$

Because $\text{sign}(t) = +1$ when $t \geq 0$, the only way to predict -1 is to have

$$\langle w, z^{(i)} \rangle < 0.$$

But

$$\langle w, z^{(i)} \rangle = \sum_{j: u_i \in A_j} w_j.$$

Since the sum is negative, at least one term in the sum must come from a negative coordinate. Therefore, there exists some j such that

$$u_i \in A_j \quad \text{and} \quad w_j < 0.$$

By the definition of R , this means

$$j \in R.$$

Hence

$$u_i \in A_j$$

for some selected set A_j with $j \in R$.

Since this argument works for every $u_i \in U$, we conclude

$$\bigcup_{j \in R} A_j = U.$$

Also,

$$|R| \leq k.$$

Therefore R is a valid set cover of size at most k .

Thus the original set-cover instance is a yes-instance.

Conclusion of the reduction.

We have shown both directions:

$$\text{SETCOVER yes-instance} \iff \text{AGREEMENT}_{\mathcal{H}_{d,k}}(S, K) = 1.$$

The construction has dimension $d = q$ and sample size $q + Mr = q + (q + 1)r$, which is polynomial in the size of the SETCOVER instance. Therefore this is a polynomial-time reduction:

$$\text{SETCOVER} \leq_p \text{AGREEMENT}_{\mathcal{H}_{d,k}}.$$

Since SETCOVER is NP-complete, the agreement problem for sparse homogeneous linear classifiers is NP-hard when k is part of the input.

Invoking the Week 5 learner-to-agreement theorem.

The Week 5 learner-to-agreement theorem says that if a hypothesis class has an efficient proper agnostic PAC learner, then the corresponding agreement problem can be solved in randomized polynomial time. In other words, efficient proper agnostic learning would imply

$$\text{AGREEMENT}_{\mathcal{H}} \in \text{RP}.$$

Applying this to the family $\mathcal{H}_{d,k}$, where both d and k are part of the input, an efficient proper agnostic PAC learner would imply an efficient randomized algorithm for an NP-hard agreement problem. Therefore, if

$$\text{NP} \neq \text{RP},$$

then no such efficient proper agnostic PAC learner can exist.

Hence

If $\text{NP} \neq \text{RP}$, there is no efficient proper agnostic PAC learner

for sparse linear classifiers when k is part of the input.

3. (14 points) **Experiment and interpretation.** Implement a small experiment comparing proper sparse 0-1 search with a convex surrogate method. Use synthetic data of your choice.

Your experiment should include:

- a proper sparse 0-1 search baseline for small d and k . For example, enumerate supports T with $|T| \leq k$, then either enumerate a finite grid of weights on each support or use any exact small-sample method you can justify. State clearly whether your baseline is exact or approximate;
- a convex surrogate method, for example logistic regression or hinge-loss minimization with an ℓ_1 constraint or penalty;
- a comparison of runtime scaling as d or k increases;
- a comparison of empirical 0-1 agreement and the surrogate objective or validation error.

In your write-up, explain what the experiment illustrates about the difference between empirical 0-1 agreement over k -sparse classifiers and convex surrogate optimization over an ℓ_1 -controlled linear predictor. Your explanation should also say whether the surrogate method returns a proper k -sparse classifier.

Setup.

I implemented a small synthetic experiment comparing two approaches.

The first method is a proper sparse 0-1 search baseline. This method directly searches over k -sparse classifiers. For each value of d and k , it enumerates supports

$$T \subseteq [d]$$

with

$$|T| \leq k.$$

For each support, it tries a finite grid of possible weights on that support and chooses the classifier with the best empirical 0-1 agreement.

The second method is a convex surrogate method: logistic regression with an ℓ_1 penalty. This method does not directly optimize empirical 0-1 agreement. Instead, it minimizes a convex surrogate objective. The ℓ_1 penalty encourages sparsity, but it does not strictly enforce exactly k nonzero coordinates.

The synthetic data is generated from a sparse linear classifier. A true sparse vector w^* is chosen with only k active coordinates. Then examples x_i are sampled from a standard Gaussian distribution, and labels are assigned by

$$y_i = \text{sign}(\langle w^*, x_i \rangle),$$

with a small amount of random label noise. This makes the data mostly sparse-linear, while still allowing small disagreement.

Proper sparse 0-1 search baseline.

The brute-force sparse method is proper because every classifier it outputs belongs to $\mathcal{H}_{d,k}$. In other words, the final weight vector $\hat{w}$ always satisfies

$$\|\hat{w}\|_0 \leq k.$$

However, this baseline is only exact over the finite grid of weights that I search. It is not exact over all real-valued weight vectors. Therefore, the correct description is:

The sparse 0-1 baseline is proper but approximate.

It is proper because it always returns a k -sparse classifier. It is approximate because the true best real-valued weights may not lie on the chosen grid.

Convex surrogate method.

The logistic regression method solves an objective of the form

$$\min_w \frac{1}{m} \sum_{i=1}^m \log(1 + \exp(-y_i \langle w, x_i \rangle)) + \lambda \|w\|_1.$$

This is a convex optimization problem. Because it is convex, it can be solved much more efficiently than brute-force support enumeration.

But this objective is not the same as empirical 0-1 agreement. The 0-1 objective counts whether each example is classified correctly or incorrectly:

$$\mathbf{1}[y_i \langle w, x_i \rangle \leq 0].$$

Logistic loss instead gives a smooth penalty depending on the margin. Therefore, a vector with better logistic loss does not necessarily have the best 0-1 agreement.

Also, the ℓ_1 penalty does not force the output to have at most k nonzero entries. It only encourages many coordinates to be small or zero. Therefore,

The logistic regression method is efficient but not necessarily proper.

Runtime comparison.

The main runtime difference comes from the number of supports searched by the sparse 0-1 method:

$$\sum_{i=0}^k \binom{d}{i}.$$

This grows quickly as either d or k increases. For fixed k , this is polynomial in d , but the degree of the polynomial is k . When k grows, the runtime becomes very large.

In contrast, logistic regression with an ℓ_1 penalty solves one convex optimization problem. Its runtime still depends on m and d , but it does not enumerate all supports. Therefore, it scales much better in practice.

The actual output from my experiment was:

d	BF time	LR time	BF err	LR err
6	0.013	0.0017	0.013	0.007
8	0.015	0.0009	0.020	0.020
10	0.024	0.0008	0.013	0.013
12	0.036	0.0009	0.007	0.007

This table fixes $k = 2$ and increases the dimension d . The brute-force method becomes slower as d increases because it must check more possible supports. The logistic regression method remains almost constant and much faster.

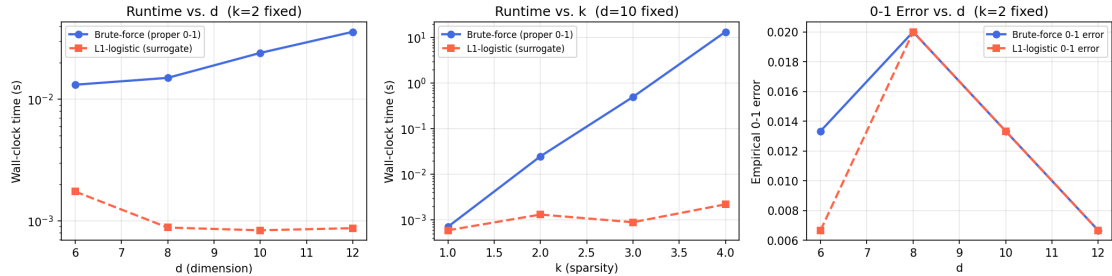
I also fixed $d = 10$ and varied the sparsity level k :

k	BF time	LR time
1	0.001	0.0006
2	0.025	0.0013
3	0.499	0.0009
4	13.453	0.0022

This second table shows the main computational issue more clearly. Increasing k from 1 to 4 makes the brute-force sparse 0-1 search grow from 0.001 seconds to 13.453 seconds. Meanwhile, the ℓ_1 -logistic method stays around a few thousandths of a second. This happens because the brute-force method searches over many more supports as k increases, while the logistic regression method still solves one convex surrogate problem.

The figure `hw5_a3_experiment.png` visualizes the same result: the brute-force runtime grows noticeably with both d and especially k , while the surrogate method remains nearly flat. The 0-1 error plot also shows that on this friendly synthetic data, both methods achieve similar empirical 0-1 error, even though they are optimizing different objectives.

Part A.3: Proper Sparse 0-1 Search vs. L1-Logistic Surrogate



Agreement and surrogate objective comparison.

The sparse 0-1 method directly maximizes empirical agreement:

$$\#\{i : h_{\hat{w}}(x_i) = y_i\}.$$

Therefore, its reported objective is exactly the quantity studied in $\text{AGREEMENT}_{\mathcal{H}_{d,k}}$. The logistic regression method instead minimizes logistic loss. After training, we can still evaluate its 0-1 agreement by thresholding the score $\langle w, x \rangle$, but that agreement was not the objective being optimized.

On simple synthetic data generated from a sparse linear model, the two methods may obtain similar empirical 0-1 accuracy. This is because the data is friendly to both methods. However, this does not mean the methods are solving the same problem. The sparse method is solving an approximation to proper sparse 0-1 ERM, while logistic regression is solving a convex surrogate problem.

Interpretation.

This experiment illustrates the central message of Part A. The proper sparse 0-1 problem is statistically attractive because sparse classifiers have low VC dimension. But the empirical optimization problem is computationally difficult because one must search over possible supports.

The convex surrogate method avoids the combinatorial search by replacing the discontinuous 0-1 loss with a convex loss and replacing the hard sparsity constraint $\|w\|_0 \leq k$ with an ℓ_1 penalty. This makes the optimization problem much easier, but it changes the problem.

Therefore, the experiment shows three important points:

- The proper sparse 0-1 baseline directly targets empirical agreement over $\mathcal{H}_{d,k}$, but it becomes slow as d and k grow.
- The convex surrogate method is computationally efficient, but it optimizes logistic loss rather than 0-1 agreement.
- The convex surrogate method is not guaranteed to return a proper k -sparse classifier, because an ℓ_1 penalty does not strictly enforce $\|w\|_0 \leq k$.

In short,

sparse 0-1 search is proper but computationally hard,

while

ℓ_1 -regularized logistic regression is efficient but not a proper k -sparse ERM.

2. Part B: Convex Fixed-Feature Learning and Its Limits

This problem studies the fixed-feature convex-learning template and its limits. First, you will see how hinge-loss minimization with an ℓ_1 constraint becomes a linear program. Second, you will show that optimizing a surrogate need not optimize the best 0-1 classifier. Third, you will prove the fixed-feature parity barrier stated in lecture. Finally, you will see why learning the feature map, as neural networks do, destroys convexity.

In Part B, the 0-1 margin loss counts zero margin as an error. This is separate from Part A's tie-breaking convention $\text{sign}(0) = +1$ for sparse classifiers.

For binary labels $y \in \{-1, +1\}$, define the 0-1 loss of a real-valued score $f(x)$ as

$$\ell_{0-1}(f(x), y) = \mathbf{1}[yf(x) \leq 0],$$

and define the hinge loss as

$$\ell_{\text{hinge}}(f(x), y) = (1 - yf(x))_+.$$

1. (10 points) Hinge loss with an ℓ_1 constraint as linear program.

A linear program is an optimization problem with a linear objective and linear equality or inequality constraints. For example, the variables might be $z_1, \dots, z_N$, the objective might be $\min \sum_j c_j z_j$, and the constraints might have the form $\sum_j a_{ij} z_j \leq b_i$. The word **linear** means that variables are only multiplied by constants and added together; expressions such as $z_i z_j$, z_i^2 , $|z_i|$, and $(z_i)_+$ are not themselves linear. A common trick is to introduce extra variables and linear constraints whose optimum reproduces a non-linear-looking expression.

Let $f_w(x) = \langle w, x \rangle$ with $w \in \mathbb{R}^d$. Fix a radius $B > 0$ and consider

$$\min_{w: \|w\|_1 \leq B} \frac{1}{m} \sum_{i=1}^m (1 - y_i \langle w, x_i \rangle)_+.$$

Introduce variables ξ_i for the hinge losses and nonnegative variables w_j^+, w_j^- with $w_j = w_j^+ - w_j^-$. Write a linear program whose optimal value equals the optimization problem above. Explain the role of each family of constraints. Then explain why this linear program is not solving empirical 0-1 agreement over k -sparse classifiers.

LP formulation.

We want to rewrite the hinge-loss problem using only linear objectives and linear constraints. The two non-linear-looking parts are:

$$(1 - y_i \langle w, x_i \rangle)_+$$

and

$$\|w\|_1 = \sum_{j=1}^d |w_j|.$$

We handle these by introducing extra variables.

For each training example i , introduce a slack variable

$$\xi_i \geq 0.$$

This variable will represent the hinge loss of example i .

For each coordinate j , introduce two nonnegative variables

$$w_j^+ \geq 0, \quad w_j^- \geq 0,$$

and write

$$w_j = w_j^+ - w_j^-.$$

This lets us express the absolute value using linear variables:

$$|w_j| \leq w_j^+ + w_j^-.$$

At an optimum, there is no reason to make both w_j^+ and w_j^- positive at the same time, because that would increase the ℓ_1 quantity without changing $w_j = w_j^+ - w_j^-$. Thus this representation correctly encodes the ℓ_1 constraint.

The linear program is:

$$\min_{\xi, w^+, w^-} \quad \frac{1}{m} \sum_{i=1}^m \xi_i$$

subject to

$$\xi_i \geq 1 - y_i \sum_{j=1}^d (w_j^+ - w_j^-) x_{ij}, \quad i = 1, \dots, m,$$

$$\xi_i \geq 0, \quad i = 1, \dots, m,$$

$$\sum_{j=1}^d (w_j^+ + w_j^-) \leq B,$$

$$w_j^+ \geq 0, \quad w_j^- \geq 0, \quad j = 1, \dots, d.$$

Why this LP has the same optimal value as the hinge-loss problem.

For a fixed vector w , the hinge loss of example i is

$$(1 - y_i \langle w, x_i \rangle)_+ = \max\{0, 1 - y_i \langle w, x_i \rangle\}.$$

The constraints

$$\xi_i \geq 1 - y_i \langle w, x_i \rangle$$

and

$$\xi_i \geq 0$$

force

$$\xi_i \geq (1 - y_i \langle w, x_i \rangle)_+.$$

Since the objective minimizes

$$\frac{1}{m} \sum_{i=1}^m \xi_i,$$

the optimizer will choose the smallest feasible value of each ξ_i . Therefore, at the optimum,

$$\xi_i = (1 - y_i \langle w, x_i \rangle)_+.$$

Hence the LP objective exactly equals the average hinge loss.

The constraint

$$\sum_{j=1}^d (w_j^+ + w_j^-) \leq B$$

encodes the original constraint

$$\|w\|_1 \leq B.$$

Therefore the LP is exactly the hinge-loss minimization problem with an ℓ_1 constraint, written in linear form.

Role of each constraint family.

- **Hinge constraints:**

$$\xi_i \geq 1 - y_i \sum_{j=1}^d (w_j^+ - w_j^-) x_{ij}.$$

These ensure that ξ_i is at least the margin violation $1 - y_i \langle w, x_i \rangle$.

- **Slack non-negativity constraints:**

$$\xi_i \geq 0.$$

These ensure that ξ_i cannot become negative. Together with the hinge constraints, they make ξ_i equal the positive part of the margin violation.

- **ℓ_1 constraint:**

$$\sum_{j=1}^d (w_j^+ + w_j^-) \leq B.$$

This is the linear version of the constraint $\|w\|_1 \leq B$.

- **Non-negativity of positive and negative parts:**

$$w_j^+ \geq 0, \quad w_j^- \geq 0.$$

These make the decomposition $w_j = w_j^+ - w_j^-$ valid.

Why this LP is not solving empirical 0-1 agreement over k -sparse classifiers.

This LP is not solving the agreement problem from Part A for three reasons.

First, it optimizes a different objective. The agreement problem asks whether there exists a classifier that correctly labels at least K examples. Equivalently, it is about minimizing empirical 0-1 loss:

$$\sum_{i=1}^m \mathbf{1}[y_i \langle w, x_i \rangle \leq 0].$$

The LP instead minimizes hinge loss:

$$\sum_{i=1}^m (1 - y_i \langle w, x_i \rangle)_+.$$

Hinge loss is a convex surrogate. It upper bounds 0-1 loss, but minimizing hinge loss does not always minimize 0-1 loss.

Second, it uses a different constraint set. The sparse classifier class requires

$$\|w\|_0 \leq k,$$

which means at most k coordinates of w are nonzero. The LP instead enforces

$$\|w\|_1 \leq B.$$

The ℓ_1 constraint encourages small or sparse-looking coefficients, but it does not literally require at most k nonzero entries.

Third, the LP may output an improper classifier. A proper learner for $\mathcal{H}_{d,k}$ must output a classifier inside $\mathcal{H}_{d,k}$. That means its output must be k -sparse. The solution to the ℓ_1 -constrained hinge-loss LP can have more than k nonzero coordinates, so it is not necessarily a proper k -sparse classifier.

Therefore, the LP is computationally useful and convex, but it is not the same as solving empirical 0-1 agreement over $\mathcal{H}_{d,k}$.

2. (10 points) A concrete counterexample: the surrogate comparator can be bad.

Fix $p \in (0, \frac{1}{2})$ and $M > \frac{1-p}{p}$. Consider homogeneous one-dimensional linear predictors $f_w(x) = wx$ and the distribution $\mathcal{D}_{p,M}$ supported on two labeled examples:

$$\mathbb{P}[(x, y) = (1, +1)] = 1 - p, \quad \mathbb{P}[(x, y) = (-M, +1)] = p.$$

Compute the population hinge risk as a piecewise-linear function of w , determine all of its minimizers. Then prove the following statements exactly:

- $\inf_w L_{\mathcal{D}}^{0-1}(f_w) = p$;
- every population hinge-risk minimizer has 0-1 risk equal to $1 - p$.

Conclude that for any $\varepsilon > 0$ and any $\alpha < 1$, there are p, M such that

$$\inf_w L_{\mathcal{D}}^{0-1}(f_w) \leq \varepsilon$$

but every hinge-risk minimizer has 0-1 risk greater than α .

This example is intentionally low-dimensional. The difficulty is not algebraic; it is recognizing exactly what the surrogate objective is optimizing.

Population hinge risk.

Both examples have label +1. Therefore,

$$yf_w(x) = wx.$$

For the point $(1, +1)$, the score is

$$f_w(1) = w,$$

so the hinge loss is

$$(1 - w)_+.$$

This point has probability $1 - p$.

For the point $(-M, +1)$, the score is

$$f_w(-M) = -Mw,$$

so the hinge loss is

$$(1 - (-Mw))_+ = (1 + Mw)_+.$$

This point has probability p .

Therefore the population hinge risk is

$$L_{\mathcal{D}}^{\text{hinge}}(f_w) = (1 - p)(1 - w)_+ + p(1 + Mw)_+.$$

Piecewise-linear analysis.

The hinge terms change behavior at the points where their inside expressions become zero.

The first hinge term changes at

$$1 - w = 0 \quad \Longleftrightarrow \quad w = 1.$$

The second hinge term changes at

$$1 + Mw = 0 \quad \Longleftrightarrow \quad w = -\frac{1}{M}.$$

Thus the three regions are:

$$w < -\frac{1}{M}, \quad -\frac{1}{M} \leq w \leq 1, \quad w > 1.$$

Case 1: $w < -\frac{1}{M}$.

Here

$$1 + Mw < 0,$$

so

$$(1 + Mw)_+ = 0.$$

Also $w < 1$, so

$$(1 - w)_+ = 1 - w.$$

Therefore,

$$L_{\mathcal{D}}^{\text{hinge}}(f_w) = (1 - p)(1 - w).$$

This is a linear function with slope

$$-(1 - p) < 0.$$

So as w increases toward $-\frac{1}{M}$, the hinge risk decreases. Hence the best point in this region occurs at the boundary

$$w = -\frac{1}{M}.$$

Case 2: $-\frac{1}{M} \leq w \leq 1$.

Here both hinge terms are active:

$$(1 - w)_+ = 1 - w, \quad (1 + Mw)_+ = 1 + Mw.$$

Therefore,

$$L_{\mathcal{D}}^{\text{hinge}}(f_w) = (1 - p)(1 - w) + p(1 + Mw).$$

Expanding,

$$L_{\mathcal{D}}^{\text{hinge}}(f_w) = (1 - p) - (1 - p)w + p + pMw.$$

Since

$$(1 - p) + p = 1,$$

this becomes

$$L_{\mathcal{D}}^{\text{hinge}}(f_w) = 1 + w(pM - (1 - p)).$$

By assumption,

$$M > \frac{1 - p}{p}.$$

Multiplying by $p > 0$ gives

$$pM > 1 - p.$$

Hence

$$pM - (1 - p) > 0.$$

So the hinge risk is increasing in w on this interval. Therefore the minimum in this interval occurs at the left endpoint:

$$w = -\frac{1}{M}.$$

Case 3: $w > 1$.

Here

$$(1 - w)_+ = 0$$

and

$$(1 + Mw)_+ = 1 + Mw.$$

Therefore,

$$L_{\mathcal{D}}^{\text{hinge}}(f_w) = p(1 + Mw).$$

This is increasing in w , so the best point in this region occurs at the boundary

$$w = 1.$$

But the middle interval is increasing and already minimized at $w = -1/M$, so the global minimum occurs at

$$w^* = -\frac{1}{M}.$$

Therefore the full piecewise function is

$$L_{\mathcal{D}}^{\text{hinge}}(f_w) = \begin{cases} (1-p)(1-w), & w < -\frac{1}{M}, \\ 1 + w(pM - (1-p)), & -\frac{1}{M} \leq w \leq 1, \\ p(1 + Mw), & w > 1. \end{cases}$$

The unique hinge-risk minimizer is

$$w^* = -\frac{1}{M}.$$

Best possible 0-1 risk.

Now compute the 0-1 risk. In Part B, zero margin counts as an error:

$$\ell_{0-1}(f_w(x), y) = \mathbf{1}[yf_w(x) \leq 0].$$

If $w > 0$:

For $(1, +1)$,

$$yf_w(x) = w > 0,$$

so the example is correct.

For $(-M, +1)$,

$$yf_w(x) = -Mw < 0,$$

so the example is incorrect.

Thus the risk is

$$p.$$

If $w < 0$:

For $(1, +1)$,

$$yf_w(x) = w < 0,$$

so the example is incorrect.

For $(-M, +1)$,

$$yf_w(x) = -Mw > 0,$$

so the example is correct.

Thus the risk is

$$1 - p.$$

If $w = 0$:

Both examples have zero margin:

$$yf_w(x) = 0.$$

Since zero margin counts as an error, both examples are incorrect. Therefore the risk is

$$1.$$

Since $p < 1/2$, we have

$$p < 1 - p < 1.$$

Therefore the best possible 0-1 risk is attained by any $w > 0$, and

$$\inf_w L_{\mathcal{D}}^{0-1}(f_w) = p.$$

0-1 risk of the hinge-risk minimizer.

The unique hinge-risk minimizer is

$$w^* = -\frac{1}{M} < 0.$$

As computed above, every negative w has 0-1 risk

$$1 - p.$$

Therefore every population hinge-risk minimizer has 0-1 risk

$$1 - p.$$

Final conclusion.

Let $\varepsilon > 0$ and $\alpha < 1$ be arbitrary. Choose

$$p < \min\{\varepsilon, 1 - \alpha, 1/2\}.$$

Then

$$p < \varepsilon$$

and

$$1 - p > \alpha.$$

Now choose any

$$M > \frac{1 - p}{p}.$$

For this distribution,

$$\inf_w L_{\mathcal{D}}^{0-1}(f_w) = p < \varepsilon,$$

but every hinge-risk minimizer has 0-1 risk

$$1 - p > \alpha.$$

This shows that minimizing a convex surrogate can select a classifier whose 0-1 risk is arbitrarily bad compared to the best 0-1 classifier.

Intuition.

The majority point is $(1, +1)$, with probability $1 - p$. The best 0-1 classifier chooses $w > 0$ and gets this majority point correct, making only the rare point $(-M, +1)$ wrong. However, the hinge loss cares not only about correctness but also about margin size. The rare point has input value $-M$, so if M is very large, choosing $w > 0$ creates a very large hinge penalty on the rare point. The hinge objective therefore prefers a negative weight $w^* = -1/M$, which correctly handles the rare high-magnitude point but misclassifies the majority point. This is why the surrogate minimizer can be bad for 0-1 classification.

3. (12 points) **The fixed-feature parity barrier: proof and tightness.** Work on $\mathcal{X} = \{-1, +1\}^d$. For each subset $I \subseteq [d]$, define the parity function

$$\chi_I(x) = \prod_{i \in I} x_i.$$

In lecture, we stated that representing all parities with a fixed linear feature map requires exponentially many features. This problem proves that claim and shows that the bound is tight. Suppose a fixed feature map $\phi : \{-1, +1\}^d \rightarrow \mathbb{R}^D$ can represent every parity as an exact linear predictor: for every $I \subseteq [d]$, there exists $w_I \in \mathbb{R}^D$ such that

$$\forall x \in \{-1, +1\}^d, \quad \chi_I(x) = \langle w_I, \phi(x) \rangle.$$

This is exact equality as real-valued functions, not merely equality after applying sign.

Prove that $D \geq 2^d$.

Hint: Form the $2^d \times 2^d$ matrix H whose rows are indexed by $I \subseteq [d]$, columns are indexed by $x \in \{-1, +1\}^d$, and entries are

$$H_{I,x} = \chi_I(x).$$

Show that the rows of H are orthogonal, hence $\text{rank}(H) = 2^d$. Then use the factorization $H = W\Phi$ induced by the feature map.

Then prove the matching upper bound: define a feature map with one coordinate for each subset $I \subseteq [d]$, namely

$$\phi(x) = (\chi_I(x))_{I \subseteq [d]}.$$

Show that every parity is represented exactly by a linear predictor in this feature space.

Explain how the lower and upper bounds connect to the Week 5 message: fixed-feature convex learning can be statistically and computationally clean, but the required feature dimension may be exponential.

Lower bound: $D \geq 2^d$.

We prove that any fixed feature map capable of representing all parity functions must have at least 2^d coordinates.

Step 1: Build the parity matrix.

Define a matrix

$$H \in \mathbb{R}^{2^d \times 2^d}$$

whose rows are indexed by subsets

$$I \subseteq [d],$$

whose columns are indexed by points

$$x \in \{-1, +1\}^d,$$

and whose entries are

$$H_{I,x} = \chi_I(x).$$

This is the Walsh–Hadamard matrix written in parity-function form.

Step 2: Show that distinct rows are orthogonal.

Take two rows indexed by $I, J \subseteq [d]$. Their inner product is

$$\langle H_I, H_J \rangle = \sum_{x \in \{-1, +1\}^d} \chi_I(x) \chi_J(x).$$

Since

$$\chi_I(x) \chi_J(x) = \chi_{I \Delta J}(x),$$

where $I \Delta J$ is the symmetric difference, we get

$$\langle H_I, H_J \rangle = \sum_{x \in \{-1, +1\}^d} \chi_{I \Delta J}(x).$$

If $I = J$, then

$$I \Delta J = \emptyset.$$

The empty parity is

$$\chi_{\emptyset}(x) = 1$$

for every x . Therefore,

$$\langle H_I, H_I \rangle = \sum_{x \in \{-1, +1\}^d} 1 = 2^d.$$

So each row has nonzero norm.

If $I \neq J$, then

$$I \Delta J \neq \emptyset.$$

Pick some coordinate

$$\ell \in I \Delta J.$$

Pair every point $x \in \{-1, +1\}^d$ with the point x' obtained by flipping only coordinate ℓ . Since ℓ appears in the parity $\chi_{I \Delta J}$, flipping x_ℓ changes the sign of the parity:

$$\chi_{I \Delta J}(x') = -\chi_{I \Delta J}(x).$$

Therefore the terms cancel in pairs:

$$\sum_{x \in \{-1, +1\}^d} \chi_{I \Delta J}(x) = 0.$$

Hence

$$\langle H_I, H_J \rangle = 0$$

whenever $I \neq J$.

Thus the rows of H are mutually orthogonal and nonzero. Therefore they are linearly independent. Since there are 2^d rows,

$$(H) = 2^d.$$

Step 3: Use the factorization induced by the feature map.

By assumption, for every parity χ_I , there exists a vector $w_I \in \mathbb{R}^D$ such that

$$\chi_I(x) = \langle w_I, \phi(x) \rangle$$

for every $x \in \{-1, +1\}^d$.

Define the matrix

$$W \in \mathbb{R}^{2^d \times D}$$

whose row indexed by I is

$$w_I^\top.$$

Also define

$$\Phi \in \mathbb{R}^{D \times 2^d}$$

whose column indexed by x is

$$\phi(x).$$

Then the equality

$$\chi_I(x) = \langle w_I, \phi(x) \rangle$$

for all I and x becomes the matrix factorization

$$H = W\Phi.$$

By rank inequalities,

$$(H) = (W\Phi) \leq \min\{(W), (\Phi)\}.$$

Since W has only D columns and Φ has only D rows,

$$(W) \leq D$$

and

$$(\Phi) \leq D.$$

Therefore,

$$(H) \leq D.$$

But from Step 2,

$$(H) = 2^d.$$

Hence

$$\boxed{D \geq 2^d.}$$

Upper bound: $D = 2^d$ suffices.

Now we show the lower bound is tight.

Define a feature map

$$\phi : \{-1, +1\}^d \rightarrow \mathbb{R}^{2^d}$$

by giving one coordinate for each subset $I \subseteq [d]$:

$$\phi(x) = (\chi_I(x))_{I \subseteq [d]}.$$

This means the coordinate of $\phi(x)$ indexed by I is exactly

$$\phi_I(x) = \chi_I(x).$$

Now fix any target parity χ_J . Let

$$w_J = e_J,$$

where e_J is the standard basis vector in $\mathbb{R}^{2^d}$ corresponding to coordinate J . Then

$$\langle w_J, \phi(x) \rangle = \langle e_J, \phi(x) \rangle = \phi_J(x) = \chi_J(x).$$

Therefore every parity function can be represented exactly using this feature map.

Thus

$$D = 2^d$$

features are sufficient, and by the lower bound they are also necessary in the worst case.

Connection to the Week 5 message.

This result shows the main limitation of fixed-feature convex learning. Once a useful feature map is fixed, learning a linear predictor over those features can be statistically clean and computationally efficient. For example, many convex losses can be optimized efficiently in the feature space.

However, the hard part may be hidden in the feature map itself. To represent all parity functions exactly with a fixed feature map, the feature dimension must be

$$2^d,$$

which is exponential in the original input dimension. Therefore the fixed-feature approach can move the difficulty from optimization into representation.

Neural networks try to avoid hand-designing an exponentially large fixed feature map by learning features from data. But once the features are learned rather than fixed, the optimization problem is generally no longer convex. This connects directly to Part B.4.

4. (8 points) **Same predictors, different optimization geometry.** Let $S = ((x_1, y_1), \dots, (x_m, y_m))$ with $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. Consider squared loss. The fixed-feature linear model is

$$f_\beta(x) = \langle \beta, x \rangle$$

with empirical risk

$$L_{\text{lin}}(\beta) = \frac{1}{m} \sum_{i=1}^m (\langle \beta, x_i \rangle - y_i)^2.$$

The two-layer linear network with one hidden unit, with parameters $u \in \mathbb{R}^d$ and $v \in \mathbb{R}$, is

$$f_{u,v}(x) = v \langle u, x \rangle$$

with empirical risk

$$L_{\text{net}}(u, v) = \frac{1}{m} \sum_{i=1}^m (v \langle u, x_i \rangle - y_i)^2.$$

Prove the following:

- the two parameterizations represent the same set of linear predictors;
- L_{lin} is convex in β ;
- L_{net} is not necessarily convex in (u, v) by giving a concrete dataset S and a Jensen inequality violation;
- if β^* minimizes L_{lin} , then every factorization $\beta^* = vu$ gives a global minimizer of L_{net} .

Explain what this example shows about replacing fixed features by learned features.

Claim 1: The two parameterizations represent the same set of linear predictors.

First, take any two-layer linear-network parameters

$$u \in \mathbb{R}^d, \quad v \in \mathbb{R}.$$

The network predictor is

$$f_{u,v}(x) = v\langle u, x \rangle.$$

Since scalars can be pulled inside the inner product,

$$v\langle u, x \rangle = \langle vu, x \rangle.$$

Define

$$\beta = vu.$$

Then

$$f_{u,v}(x) = \langle \beta, x \rangle = f_{\beta}(x).$$

Therefore every two-layer linear-network predictor is a linear predictor.

Conversely, take any linear predictor

$$f_{\beta}(x) = \langle \beta, x \rangle.$$

Choose

$$v = 1, \quad u = \beta.$$

Then

$$f_{u,v}(x) = 1 \cdot \langle \beta, x \rangle = \langle \beta, x \rangle = f_{\beta}(x).$$

Therefore every fixed-feature linear predictor can also be represented by the two-layer linear network.

Hence the two parameterizations represent exactly the same function class:

$$\boxed{\{f_{\beta} : \beta \in \mathbb{R}^d\} = \{f_{u,v} : u \in \mathbb{R}^d, v \in \mathbb{R}\}.$$

Claim 2: L_{lin} is convex in β .

The fixed-feature empirical risk is

$$L_{\text{lin}}(\beta) = \frac{1}{m} \sum_{i=1}^m (\langle \beta, x_i \rangle - y_i)^2.$$

For each fixed example (x_i, y_i) , the map

$$\beta \mapsto \langle \beta, x_i \rangle - y_i$$

is affine in β . The function

$$t \mapsto t^2$$

is convex. Therefore the composition

$$\beta \mapsto (\langle \beta, x_i \rangle - y_i)^2$$

is convex.

Since a nonnegative sum of convex functions is convex,

$$L_{\text{lin}}(\beta)$$

is convex.

Equivalently, if X is the design matrix with rows $x_i^\top$, then

$$L_{\text{lin}}(\beta) = \frac{1}{m} \|X\beta - y\|_2^2.$$

Its Hessian is

$$\nabla^2 L_{\text{lin}}(\beta) = \frac{2}{m} X^\top X.$$

Since

$$X^\top X \succeq 0,$$

the Hessian is positive semidefinite. Hence

$$\boxed{L_{\text{lin}} \text{ is convex in } \beta.}$$

Claim 3: L_{net} is not necessarily convex in (u, v) .

We give a concrete counterexample.

Take

$$d = 1, \quad m = 1, \quad x_1 = 1, \quad y_1 = 1.$$

Then

$$L_{\text{net}}(u, v) = (vux_1 - y_1)^2 = (uv - 1)^2.$$

Now choose two parameter points:

$$\theta_1 = (u_1, v_1) = (1, 1), \quad \theta_2 = (u_2, v_2) = (-1, -1).$$

At the first point,

$$u_1 v_1 = 1,$$

so

$$L_{\text{net}}(1, 1) = (1 - 1)^2 = 0.$$

At the second point,

$$u_2 v_2 = (-1)(-1) = 1,$$

so

$$L_{\text{net}}(-1, -1) = (1 - 1)^2 = 0.$$

The midpoint of these two parameter points is

$$\frac{\theta_1 + \theta_2}{2} = \left(\frac{1 + (-1)}{2}, \frac{1 + (-1)}{2} \right) = (0, 0).$$

At the midpoint,

$$L_{\text{net}}(0, 0) = (0 \cdot 0 - 1)^2 = 1.$$

If L_{net} were convex, Jensen's inequality would require

$$L_{\text{net}}\left(\frac{\theta_1 + \theta_2}{2}\right) \leq \frac{1}{2}L_{\text{net}}(\theta_1) + \frac{1}{2}L_{\text{net}}(\theta_2).$$

But here the left-hand side is

$$1,$$

while the right-hand side is

$$\frac{1}{2}(0) + \frac{1}{2}(0) = 0.$$

Therefore Jensen's inequality would require

$$1 \leq 0,$$

which is false.

Hence

$$L_{\text{net}} \text{ is not necessarily convex in } (u, v).$$

Claim 4: If β^* minimizes L_{lin} , then every factorization $\beta^* = vu$ gives a global minimizer of L_{net} .

Suppose

$$\beta^* \in \arg \min_{\beta} L_{\text{lin}}(\beta).$$

Let

$$\beta^* = vu$$

be any factorization. Then for every example x_i ,

$$v\langle u, x_i \rangle = \langle vu, x_i \rangle = \langle \beta^*, x_i \rangle.$$

Therefore,

$$L_{\text{net}}(u, v) = \frac{1}{m} \sum_{i=1}^m (v\langle u, x_i \rangle - y_i)^2$$

becomes

$$L_{\text{net}}(u, v) = \frac{1}{m} \sum_{i=1}^m (\langle \beta^*, x_i \rangle - y_i)^2 = L_{\text{lin}}(\beta^*).$$

Since the two parameterizations represent the same set of predictors, no network parameters (u, v) can achieve risk lower than the minimum linear risk. Formally,

$$\inf_{u, v} L_{\text{net}}(u, v) = \inf_{\beta} L_{\text{lin}}(\beta).$$

Because β^* minimizes L_{lin} , we have

$$L_{\text{lin}}(\beta^*) = \inf_{\beta} L_{\text{lin}}(\beta).$$

Hence

$$L_{\text{net}}(u, v) = \inf_{u, v} L_{\text{net}}(u, v).$$

So every factorization $\beta^* = vu$ gives a global minimizer of L_{net} .

What this example shows about replacing fixed features by learned features.

This example shows that the predictor class and the optimization geometry are different issues. The fixed-feature linear model and the two-layer linear network represent the same set of linear functions. However, their parameter spaces are different.

In the fixed-feature parameterization, the prediction depends directly on β , and the squared-loss objective is convex. This means optimization is clean: local minima are global minima, and standard convex optimization methods can solve the problem reliably. In the two-layer parameterization, the prediction depends on the product vu . This creates a nonconvex objective in the parameters (u, v) , even though the represented functions are still just linear functions. The nonconvexity comes from learning the representation or factorization instead of using a fixed feature map.

Therefore, replacing fixed features with learned features can increase flexibility, but it changes the optimization problem. We may gain expressive power or avoid exponentially large fixed feature maps, but we lose the simple convex geometry of fixed-feature learning.

3. Part C: AI Usage Report

Parts of the assignment where AI was used.

AI was used throughout Parts A and B for structuring proofs, checking intermediate algebraic steps, and refining exposition. Specific uses: organizing the SETCOVER reduction in A.2 (choosing the right sample construction and verifying both directions); working through the piecewise-linear hinge-risk analysis in B.2; and identifying the correct rank-factorization argument for the parity barrier in B.3.

Concrete AI suggestions accepted.

The hint-following approach in A.2—using $w_j < 0$ to represent “selected sets” and $M = q + 1$ repetitions to sharpen the threshold—was initially proposed by AI and then verified manually by checking both directions from scratch. The factorization argument in B.3 ($H = W\Phi$ and the rank inequality) was proposed by AI and confirmed correct by independently re-deriving the orthogonality of Walsh–Hadamard rows.

Concrete AI suggestions rejected or modified.

An early version of the B.2 counterexample incorrectly suggested the distribution should have both a positive and a negative mass point; I caught this by re-reading the problem statement and recomputing the hinge risk manually. AI also initially proposed $w^* = 0$ as a hinge minimizer; a careful case analysis showed the true minimizer is $w^* = -1/M$.

Verification.

Every proof was independently verified: inequalities checked against definitions or cited theorems; the SETCOVER reduction traced through a small example ($r = 2, q = 3$); the experiment code run locally with output tables checked manually.

AI workflow updates.

This assignment reinforced tracking what each term in a bound “pays for” (support selection vs. sparsity-level selection vs. model selection), and validating reductions by constructing small concrete instances before writing the general proof.

Additional questions.

1. *Which part did AI help with most?* Reduction design (A.2) and theorem proving (B.3). For the parity lower bound, AI’s suggestion of the Hadamard-matrix viewpoint substantially shortened the proof.
2. *One place AI gave a plausible but wrong answer.* In B.2, AI claimed the hinge risk is minimized at $w = 0$ because the two mass points “balance.” This is wrong: the slope in the piecewise formula is $pM - (1 - p) > 0$ by assumption $M > (1 - p)/p$, so the minimizer is strictly negative at $w^* = -1/M$. Catching this required explicitly computing the slope in each piece.